

Infera

Complete Technical Documentation

AI Agent Monitoring & Anomaly Detection Platform

Final Year Project — Phase 1 (Sem 7) + Phase 2 (Sem 8)

Document Version 1.0 — June 2026

Reference this document to continue Phase 2 development from any point

1. Project Overview

Infera is a real-time monitoring and anomaly detection platform built for autonomous AI agents powered by Large Language Models. Traditional APM tools (Datadog, New Relic) are designed for deterministic software — they cannot capture LLM-specific failure modes such as token consumption spikes, infinite reasoning loops, tool failure cascades, or behavioral drift over time. Infera fills this gap.

1.1 Problem Statement

- AI agents can enter infinite thought-action-observation loops, consuming API tokens and budget with no progress.
- Token consumption is non-deterministic; a single rogue session can exhaust a daily API budget in seconds.
- Tool failures cascade silently — the agent retries endlessly and produces incorrect output without raising an exception.
- No open-source platform provides a unified observability layer designed specifically for LLM agent behavior.

1.2 Solution Summary

- Real-time telemetry ingestion via REST API — every agent action emits a structured JSON event.
- Isolation Forest unsupervised ML model detects anomalies across a 10-dimensional feature space.
- Agent Reliability Score — an original composite metric predicting agent health before failures occur.
- React dashboard with live charts, anomaly alerts, and tool invocation graph visualization.

1.3 Two-Phase Roadmap

	Phase 1 — Semester 7 (3 credits)	Phase 2 — Semester 8
Goal	Complete, deployable MVP — demonstrated at viva	Enterprise-scale distributed system + paper
Architecture	Single-node: FastAPI + PostgreSQL + React	Kafka + Redis + Prometheus + Grafana + K8s
ML	Isolation Forest + Reliability Score (offline)	Streaming anomaly detection, model versioning
Agents	3 simulated agents (built-in)	Real LLM agents via Python SDK
Deployment	Docker Compose — one command on any laptop	Kubernetes cluster
Auth	JWT, single user role	Multi-user, RBAC (admin/operator/viewer)
Research	Evaluation dataset + benchmarks collected	Paper written and submitted

1.4 Key Research Contributions

- **Original:** Agent Reliability Score — original composite metric (tool success 40%, token efficiency 20%, latency 20%, loop frequency 20%) producing a 0–100 predictive health index updated after every action.
- **ML:** Behavioral drift detection via Isolation Forest on multi-dimensional agent telemetry — no labeled anomaly data required.
- **Schema:** Standardized telemetry schema for LLM agent execution events — reusable across any agent architecture.

2. Technology Stack

2.1 Phase 1 Stack

Component	Technology	Version	Rationale
Backend API	FastAPI	0.111+	Async, auto-generates Swagger docs, Python ML libraries run in-process — no separate ML microservice needed
Database	PostgreSQL	15+	ACID, JSONB for telemetry storage, window functions for rolling stats — sufficient for Phase 1 volume
ORM / Migrations	SQLAlchemy + Alembic	2.0+	Async ORM for FastAPI; Alembic tracks schema changes as versioned migration files
ML	Scikit-Learn	1.4+	Isolation Forest, LOF, StandardScaler, joblib model persistence
Data Processing	Pandas + NumPy	2.0+	Feature engineering, rolling window statistics on telemetry streams
Task Scheduler	APScheduler	3.10+	In-process periodic jobs (model retrain, stats aggregation). No broker needed — lighter than Celery
Auth	python-jose + passlib	latest	JWT HS256 token generation; bcrypt password hashing
Frontend	React	18+	Component-based, efficient re-rendering on polling updates
Charts	Recharts	2.12+	Native React chart library; LineChart, RadialBarChart, BarChart — real-time data support
HTTP Client	Axios	1.7+	API calls from React with request/response interceptors for JWT attachment and 401 redirect
Containerisation	Docker + Docker Compose	latest	Single-command deployment; eliminates environment mismatch during viva

2.2 Phase 2 Additions

Component	Technology	Purpose
Message Broker	Apache Kafka (Confluent)	Decouples agent telemetry ingestion from backend processing; handles thousands of events/second
Cache / Pub-Sub	Redis	Caches dashboard aggregates (5 s TTL); pub-sub for pushing scored events to SSE endpoint
Metrics Store	Prometheus	Scrapes /metrics endpoint every 15 s; stores time-series for Grafana
Visualisation	Grafana	Infrastructure dashboards (Kafka lag, DB query times) complementing the custom React UI
Agent SDK	Python package (pip install)	Decorator and context-manager API for auto-instrumenting real LangChain or custom agents
Orchestration	Kubernetes	Horizontal pod autoscaling; separate deployments for backend, Kafka consumer, notification service

3. Repository Structure

Monorepo. All paths are relative to the project root Infera/.

```
Infera/
├── backend/
│   ├── app/
│   │   └── main.py # FastAPI app entry - registers routers,
│   ├── startup events
│   │   ├── config.py # Pydantic BaseSettings - reads all env vars
│   │   ├── database.py # Async SQLAlchemy engine + session factory
│   │   └── models/ # SQLAlchemy ORM models (one file per table)
│   │       ├── user.py
│   │       ├── agent.py
│   │       ├── session.py
│   │       ├── telemetry.py
│   │       ├── alert.py
│   │       └── reliability.py
│   ├── schemas/ # Pydantic request/response models
│   │   ├── auth.py # LoginRequest, TokenResponse
│   │   ├── agent.py # AgentCreate, AgentRead, AgentUpdate
│   │   ├── telemetry.py # TelemetryEvent, TelemetryResponse
│   │   └── alert.py # AlertRead, AlertStats
│   ├── routers/ # FastAPI APIRouter files
│   │   ├── auth.py
│   │   ├── agents.py
│   │   ├── sessions.py
│   │   ├── telemetry.py
│   │   ├── anomalies.py
│   │   ├── dashboard.py
│   │   ├── simulator.py
│   │   └── ml.py
│   ├── ml/ # Machine learning pipeline
│   │   ├── feature_engineering.py # Raw event → 10-dim feature vector
│   │   ├── isolation_forest.py # IF training, scoring, persistence
│   │   ├── lof_baseline.py # LOF model for evaluation comparison
│   │   ├── reliability_score.py # Composite score formula
│   │   └── model_store.py # joblib save/load + atomic hot-swap
│   ├── simulator/ # Built-in simulated agents
│   │   ├── base_agent.py # Abstract base class
│   │   ├── customer_support.py # Agent A001
│   │   ├── research_agent.py # Agent A002
│   │   ├── sales_agent.py # Agent A003
│   │   └── anomaly_injector.py # On-demand anomaly injection
│   └── services/
│       ├── alert_service.py # Alert generation logic + threshold checks
│       ├── scheduler.py # APScheduler job definitions
│       └── auth_service.py # JWT create/verify + password hashing
├── alembic/ # DB migration versions
│   └── versions/
├── alembic.ini
├── requirements.txt
├── Dockerfile
└── frontend/
    ├── src/
    │   ├── components/
    │   │   ├── Layout/ # Sidebar + TopBar + PageWrapper
    │   │   ├── AgentCard/ # Per-agent status card with score badge
    │   │   ├── Charts/ # TokenChart, LatencyChart, CostChart
    │   │   ├── AnomalyLog/ # Live scrolling alert table
    │   │   ├── ReliabilityGauge/ # Semicircle gauge + component breakdown
    │   │   └── ToolGraph/ # Tool invocation DAG (Recharts or D3)
    │   └── pages/
    │       ├── Login.jsx
    │       ├── Dashboard.jsx # Main overview - KPIs + agent grid + log
    │       └── AgentDetail.jsx # Per-agent deep-dive page
```

```

├── AnomalyHistory.jsx      # Filterable historical alert table
├── Settings.jsx           # Per-agent threshold editor
├── hooks/
│   ├── useAgents.js       # Polls /dashboard/agents/active every 5 s
│   └── useTelemetry.js    # Polls telemetry for active agent detail
├── api/
│   ├── client.js          # Axios instance + JWT interceptors
│   └── context/
│       └── AuthContext.jsx # Login state, token storage, logout
├── package.json
├── Dockerfile
├── docker-compose.yml
├── docker-compose.dev.yml # Mounts source for hot-reload (no Nginx)
├── .env.example
└── README.md

```

4. Database Schema (PostgreSQL)

Managed via SQLAlchemy async ORM + Alembic migrations. Total content width on A4: 9026 DXA. All migration files live in backend/alembic/versions/.

4.1 users

Column	Type	Constraints	Description
id	UUID	PK, DEFAULT gen_random_uuid()	Unique user identifier
username	VARCHAR(100)	UNIQUE NOT NULL	Login handle
email	VARCHAR(255)	UNIQUE NOT NULL	User email
hashed_password	VARCHAR(255)	NOT NULL	bcrypt hash — never store plaintext
is_active	BOOLEAN	DEFAULT TRUE	Soft-delete flag
created_at	TIMESTAMP	DEFAULT NOW()	Account creation timestamp

4.2 agents

Column	Type	Constraints	Description
id	VARCHAR(50)	PK	Agent ID, e.g. 'A001'
name	VARCHAR(100)	NOT NULL	Display name, e.g. 'Customer Support Agent'
type	VARCHAR(50)	NOT NULL	customer_support research sales
description	TEXT	NULLABLE	Free-text purpose description
token_budget	INTEGER	DEFAULT 10000	Maximum tokens per session before alert
latency_threshold_ms	FLOAT	DEFAULT 3000.0	Latency (ms) above which WARNING fires
failure_threshold	FLOAT	DEFAULT 0.30	Failure rate above which WARNING fires (0–1)
loop_threshold	INTEGER	DEFAULT 10	loop_count above which CRITICAL fires
is_active	BOOLEAN	DEFAULT TRUE	Whether agent is currently monitored
owner_id	UUID	FK → users.id	Owning user
created_at	TIMESTAMP	DEFAULT NOW()	Registration time

4.3 sessions

Column	Type	Constraints	Description
id	VARCHAR(50)	PK	Session ID, e.g. 'S1001'
agent_id	VARCHAR(50)	FK → agents.id NOT NULL	Owning agent
started_at	TIMESTAMP	DEFAULT NOW()	Session start time

Column	Type	Constraints	Description
ended_at	TIMESTAMP	NULLABLE	NULL while session is active
status	VARCHAR(20)	DEFAULT 'active'	active completed failed terminated
total_tokens	INTEGER	DEFAULT 0	Cumulative tokens used — updated on each event
total_cost_usd	FLOAT	DEFAULT 0.0	Running cost estimate (tokens * cost_per_1k / 1000)
total_tool_calls	INTEGER	DEFAULT 0	Total tool invocations in session
failed_tool_calls	INTEGER	DEFAULT 0	Failed invocations — used in Reliability Score

4.4 telemetry_events

Highest-volume table. Expected 10–100 rows per session. Index on (agent_id, timestamp) is critical for dashboard queries.

Column	Type	Constraints	Description
id	BIGSERIAL	PK	Auto-increment event ID
agent_id	VARCHAR(50)	FK, INDEX	Source agent — indexed for fast per-agent queries
session_id	VARCHAR(50)	FK → sessions.id	Source session
timestamp	TIMESTAMP	NOT NULL, INDEX	When the agent action occurred (agent-side clock)
tokens_used	INTEGER	NOT NULL	Tokens consumed in this single action
tool_name	VARCHAR(100)	NULLABLE	Tool invoked; NULL for pure LLM reasoning steps
latency_ms	FLOAT	NOT NULL	End-to-end action latency in milliseconds
status	VARCHAR(20)	NOT NULL	SUCCESS FAILURE TIMEOUT
loop_count	INTEGER	DEFAULT 1	Current iteration counter within session
prompt_length	INTEGER	NULLABLE	Character count of input prompt
response_length	INTEGER	NULLABLE	Character count of model response
error_message	TEXT	NULLABLE	Error detail when status = FAILURE
raw_payload	JSONB	NULLABLE	Full raw JSON event for audit replay
anomaly_score	FLOAT	NULLABLE	Isolation Forest score (lower = more anomalous)
is_anomaly	BOOLEAN	DEFAULT FALSE	True when IF score passes threshold
created_at	TIMESTAMP	DEFAULT NOW()	DB insertion time (may lag timestamp slightly)

4.5 anomaly_alerts

Column	Type	Constraints	Description
id	BIGSERIAL	PK	Alert ID

Column	Type	Constraints	Description
agent_id	VARCHAR(50)	FK, INDEX	Affected agent
session_id	VARCHAR(50)	FK	Session during which alert fired
telemetry_event_id	BIGINT	FK → telemetry_events	The event that triggered this alert
alert_type	VARCHAR(50)	NOT NULL	token_spike infinite_loop high_latency tool_failure_cascade behavioral_drift
severity	VARCHAR(20)	NOT NULL	INFO WARNING CRITICAL
anomaly_score	FLOAT	NOT NULL	Raw IF score that triggered the alert
description	TEXT	NOT NULL	Human-readable message, e.g. 'Token spike: 850 vs baseline 120'
threshold_value	FLOAT	NULLABLE	The configured threshold that was breached
actual_value	FLOAT	NULLABLE	The actual value observed
is_acknowledged	BOOLEAN	DEFAULT FALSE	Operator has acknowledged this alert
created_at	TIMESTAMP	DEFAULT NOW()	Alert generation time

4.6 agent_reliability_scores

Column	Type	Constraints	Description
id	BIGSERIAL	PK	Score record
agent_id	VARCHAR(50)	FK, INDEX	Agent being scored
session_id	VARCHAR(50)	FK	Session context for this score
score	FLOAT	NOT NULL	Composite score 0–100
tool_success_rate	FLOAT	NOT NULL	Successful calls / total calls
token_efficiency	FLOAT	NOT NULL	Expected tokens / actual tokens (capped 1.0)
latency_score	FLOAT	NOT NULL	Normalized inverse latency vs baseline
loop_frequency_score	FLOAT	NOT NULL	Inverse of average loop count
risk_level	VARCHAR(20)	NOT NULL	LOW MEDIUM HIGH CRITICAL
predicted_failure_prob	FLOAT	NOT NULL	Estimated prob of failure in next 10 calls (0–1)
calculated_at	TIMESTAMP	DEFAULT NOW()	Calculation time

4.7 ml_model_metadata

Column	Type	Constraints	Description
id	SERIAL	PK	Model record
model_type	VARCHAR(50)	NOT NULL	isolation_forest lof
agent_id	VARCHAR(50)	NULLABLE	NULL = global model, non-null = per-agent model

Column	Type	Constraints	Description
model_path	TEXT	NOT NULL	Absolute path to joblib file, e.g. /app/models/A001_if.pkl
trained_on_count	INTEGER	NOT NULL	Number of events in the training set
contamination	FLOAT	NOT NULL	IF contamination parameter used
precision_score	FLOAT	NULLABLE	Precision on validation set (from eval run)
recall_score	FLOAT	NULLABLE	Recall on validation set
f1_score	FLOAT	NULLABLE	F1 on validation set
trained_at	TIMESTAMP	DEFAULT NOW()	Training timestamp
is_active	BOOLEAN	DEFAULT TRUE	Whether this model is currently loaded in memory

4.8 Required Indexes

```
-- Run these after initial migration
CREATE INDEX idx_tel_agent_time ON telemetry_events(agent_id, timestamp DESC);
CREATE INDEX idx_tel_session ON telemetry_events(session_id);
CREATE INDEX idx_tel_anomaly ON telemetry_events(is_anomaly) WHERE is_anomaly =
TRUE;
CREATE INDEX idx_alert_agent ON anomaly_alerts(agent_id, created_at DESC);
CREATE INDEX idx_alert_unacked ON anomaly_alerts(is_acknowledged) WHERE
is_acknowledged = FALSE;
CREATE INDEX idx_score_agent ON agent_reliability_scores(agent_id,
calculated_at DESC);
```

5. REST API Reference

Base URL (dev): <http://localhost:8000/api/v1> | Swagger UI: <http://localhost:8000/docs> | All endpoints except `/auth/login` and `/auth/register` require Authorization: Bearer <token>

5.1 Auth — /auth

Method	Endpoint	Auth	Description
POST	/auth/register	None	Create new user. Body: {username, email, password}
POST	/auth/login	None	Returns access_token + refresh_token
POST	/auth/refresh	Refresh	Exchange refresh token for new access token
GET	/auth/me	Bearer	Return current user profile

Login Request / Response

```
// POST /auth/login
Body:    { "username": "admin", "password": "secret123" }

// Response 200
{
  "access_token": "eyJhbGciOi...",
  "refresh_token": "eyJhbGciOi...",
  "token_type": "bearer",
  "expires_in": 1800
}
```

5.2 Agents — /agents

Method	Endpoint	Auth	Description
GET	/agents/	Bearer	List all agents owned by current user
POST	/agents/	Bearer	Register a new agent. Body: AgentCreate schema
GET	/agents/{id}	Bearer	Get agent config + current thresholds
PUT	/agents/{id}	Bearer	Update token_budget, thresholds. Body: AgentUpdate
DELETE	/agents/{id}	Bearer	Soft-delete (set is_active=false)
GET	/agents/{id}/sessions	Bearer	List sessions for this agent (paginated)
GET	/agents/{id}/reliability	Bearer	Latest Reliability Score record for this agent

5.3 Sessions — /sessions

Method	Endpoint	Auth	Description
POST	/sessions/start	Bearer	Start a new session. Body: {agent_id, session_id}
PUT	/sessions/{id}/end	Bearer	Mark session completed/failed. Body: {status}
GET	/sessions/{id}	Bearer	Session summary: total tokens, cost, tool call counts
GET	/sessions/{id}/events	Bearer	Paginated list of telemetry events in this session

Method	Endpoint	Auth	Description
GET	/sessions/{id}/tool-graph	Bearer	Ordered tool invocation sequence for graph rendering

5.4 Telemetry — /telemetry

Method	Endpoint	Auth	Description
POST	/telemetry/ingest	Bearer	Ingest one event; returns anomaly_score + is_anomaly immediately
POST	/telemetry/batch	Bearer	Ingest up to 100 events in one call
GET	/telemetry/{agent_id}	Bearer	Recent events for an agent. Query: ?limit=50&offset=0
GET	/telemetry/{agent_id}/stats	Bearer	Aggregated stats: avg tokens, avg latency, success rate

POST /telemetry/ingest — Full Schema

```
// Request Body
{
  "agent_id":      "A001",           // required
  "session_id":    "S1001",          // required
  "timestamp":     "2026-06-15T10:30:00Z", // required, ISO 8601
  "tokens_used":   125,              // required, integer
  "tool_name":     "web_search",      // nullable
  "latency_ms":    1250.0,           // required, float
  "status":        "SUCCESS",         // required: SUCCESS|FAILURE|TIMEOUT
  "loop_count":    1,                // optional, default 1
  "prompt_length": 340,              // optional
  "response_length": 280,            // optional
  "error_message": null              // optional
}

// Response 200
{
  "event_id":      42,
  "anomaly_score": 0.12,              // IF score; < -0.5 = anomalous
  "is_anomaly":    false,
  "alert_generated": false,
  "alert_id":      null,              // set if alert_generated=true
  "reliability_score": 87.4           // updated score after this event
}
```

5.5 Anomalies — /anomalies

Method	Endpoint	Auth	Description
GET	/anomalies/	Bearer	All alerts, filterable by ?agent_id=&severity=&type=&limit=
GET	/anomalies/{agent_id}	Bearer	Alerts for a specific agent (paginated)
PUT	/anomalies/{id}/acknowledge	Bearer	Set is_acknowledged=true
GET	/anomalies/stats	Bearer	Counts by type and severity for the last 24 h
GET	/anomalies/unacknowledged	Bearer	All alerts where is_acknowledged=false

5.6 Dashboard — /dashboard

Method	Endpoint	Auth	Description
GET	/dashboard/overview	Bearer	KPIs: active agent count, total alerts 24h, avg reliability score, total cost today
GET	/dashboard/agents/active	Bearer	All active agents with their latest reliability score + risk_level
GET	/dashboard/metrics/token-usage	Bearer	Token usage time-series per agent. Query: ?window=1h 6h 24h
GET	/dashboard/metrics/costs	Bearer	Cumulative USD cost per agent over selected window
GET	/dashboard/metrics/latency	Bearer	Average latency per tool across all agents
GET	/dashboard/alerts/recent	Bearer	Last 20 alerts across all agents (unfiltered)

5.7 Simulator — /simulator

Method	Endpoint	Auth	Description
POST	/simulator/start	Bearer	Start all three built-in agents emitting telemetry
POST	/simulator/stop	Bearer	Stop all running simulators gracefully
GET	/simulator/status	Bearer	Returns {running: bool, agents: [{id, status, event_count}]}
POST	/simulator/inject-anomaly	Bearer	Inject specific anomaly into a running agent (see schema below)

```
// POST /simulator/inject-anomaly
{
  "agent_id":      "A001",
  "anomaly_type":  "token_spike", // token_spike|infinite_loop|high_latency
                                   // |tool_failure_cascade|behavioral_drift
  "duration_events": 3           // how many events to inject anomalously
}
```

5.8 ML — /ml

Method	Endpoint	Auth	Description
POST	/ml/retrain	Bearer	Trigger immediate model retraining (APScheduler job also runs every 30 min)
GET	/ml/model/stats	Bearer	Current model metadata: trained_on_count, contamination, f1_score, trained_at
POST	/ml/predict	Bearer	Score a single event without storing it — useful for testing

6. Machine Learning Pipeline

The ML pipeline lives entirely in backend/app/ml/. It runs in-process with FastAPI — no separate microservice required in Phase 1.

6.1 Feature Engineering (feature_engineering.py)

Every telemetry event is converted to a 10-dimensional feature vector before scoring. Features capture both the instantaneous event and rolling behavioral context.

#	Feature Name	Source	Transformation	Why It Matters
1	tokens_used	Raw event	Pass-through integer	Absolute token consumption per action
2	tokens_zscore	Computed	Z-score vs agent rolling mean/std (window=50 events)	Normalises to agent baseline — catches relative spikes
3	latency_ms	Raw event	Pass-through float	Absolute latency
4	latency_zscore	Computed	Z-score vs agent latency rolling mean	Catches latency spikes relative to norm
5	loop_count	Raw event	Pass-through integer	Direct loop iteration — primary loop-detection signal
6	is_failure	Raw event	1 if status in {FAILURE, TIMEOUT} else 0	Binary failure flag
7	tokens_per_ms	Computed	tokens_used / latency_ms	Token efficiency; drops during high-latency states
8	rolling_fail_rate	Computed	Failures / calls in last 10 events	Short-window failure trend; cascades raise this fast
9	rolling_avg_tokens	Computed	Mean tokens in last 10 events	Local context for spike detection
10	session_age_ratio	Computed	events_in_session / MAX_SESSION_LEN	Position in session lifecycle

```
# feature_engineering.py (pseudocode)
def extract_features(event: dict, agent_history: deque) -> np.ndarray:
    tokens      = event['tokens_used']
    latency     = event['latency_ms']
    hist_tok    = [e['tokens_used'] for e in agent_history]
    hist_lat    = [e['latency_ms'] for e in agent_history]

    tok_mean, tok_std = np.mean(hist_tok), np.std(hist_tok) + 1e-6
    lat_mean, lat_std = np.mean(hist_lat), np.std(hist_lat) + 1e-6

    last10      = list(agent_history)[-10:]
    fail_rate   = sum(1 for e in last10 if e['status'] != 'SUCCESS') / len(last10)
    avg_tok_10  = np.mean([e['tokens_used'] for e in last10])

    return np.array([
        tokens,
        (tokens - tok_mean) / tok_std,      # tokens_zscore
        latency,
        (latency - lat_mean) / lat_std,     # latency_zscore
        event['loop_count'],
        1 if event['status'] != 'SUCCESS' else 0,
```

```

        tokens / (latency + 1e-6),          # tokens_per_ms
        fail_rate,
        avg_tok_10,
        event['session_event_count'] / MAX_SESSION_LEN,
    ])

```

6.2 Isolation Forest (isolation_forest.py)

```

# isolation_forest.py
from sklearn.ensemble import IsolationForest
from sklearn.preprocessing import StandardScaler
import joblib, os

IF_CONFIG = {
    'n_estimators' : 100,      # number of isolation trees
    'contamination': 0.05,    # expected anomaly fraction (5%)
    'random_state' : 42,      # reproducibility
    'max_samples'  : 'auto',  # min(256, n_samples)
    'n_jobs'       : -1,      # all CPU cores
}

class IFModel:
    def __init__(self):
        self.scaler = StandardScaler()
        self.model = IsolationForest(**IF_CONFIG)
        self.fitted = False

    def train(self, feature_matrix: np.ndarray):
        # Require at least 200 events for meaningful training
        if len(feature_matrix) < 200:
            raise ValueError('Need >= 200 events to train')
        X = self.scaler.fit_transform(feature_matrix)
        self.model.fit(X)
        self.fitted = True

    def score(self, feature_vector: np.ndarray) -> tuple[float, bool]:
        # Returns (anomaly_score, is_anomaly)
        # score_samples: more negative = more anomalous
        X = self.scaler.transform(feature_vector.reshape(1, -1))
        score = float(self.model.score_samples(X)[0])
        label = int(self.model.predict(X)[0]) # -1=anomaly, 1=normal
        return score, label == -1

    def save(self, path: str):
        joblib.dump({'scaler': self.scaler, 'model': self.model}, path + '.tmp')
        os.replace(path + '.tmp', path) # atomic swap - no downtime

    @classmethod
    def load(cls, path: str):
        obj = cls()
        data = joblib.load(path)
        obj.scaler, obj.model, obj.fitted = data['scaler'], data['model'], True
        return obj

```

6.3 Anomaly Score → Alert Severity Mapping

IF Score Range	is_anomaly	Severity	Dashboard Action
Above -0.3	False	—	No alert; normal operation logged
-0.3 to -0.5	True	INFO	Logged to anomaly_alerts; shown in log panel

IF Score Range	is_anomaly	Severity	Dashboard Action
-0.5 to -0.7	True	WARNING	Dashboard notification badge + amber card highlight
Below -0.7	True	CRITICAL	Browser popup + persistent red banner + sound

IF score_samples output ranges approximately -0.8 to +0.2 in practice. The thresholds above were calibrated on synthetic data; tune via the ANOMALY_CONTAMINATION_RATE env var and re-evaluate.

6.4 LOF Baseline (lof_baseline.py)

Used only during evaluation — NOT in the production scoring path. Provides a comparison model for the research paper.

```
from sklearn.neighbors import LocalOutlierFactor

LOF_CONFIG = {
    'n_neighbors' : 20,
    'contamination': 0.05,
    'novelty'      : True,    # enables predict() on unseen data
    'n_jobs'       : -1,
}

# Train exactly like IF — same feature matrix, same scaler
# At evaluation time, compare IF vs LOF precision/recall/F1
```

6.5 Agent Reliability Score (reliability_score.py)

Original research contribution. Computed after every telemetry event and written to agent_reliability_scores.

```
def compute_reliability_score(session_stats: dict) -> dict:
    ''
    session_stats keys:
        successful_calls, total_calls,
        expected_tokens, actual_tokens,
        avg_latency, baseline_latency,
        avg_loop_count
    ''
    # Component 1 — Tool Success Rate (40%)
    tool_sr = session_stats['successful_calls'] / max(session_stats['total_calls'],
1)

    # Component 2 — Token Efficiency (20%)
    # expected_tokens = agent type average; capped at 1.0 (can't be more efficient
than expected)
    tok_eff = min(session_stats['expected_tokens'] /
max(session_stats['actual_tokens'], 1), 1.0)

    # Component 3 — Latency Score (20%)
    # Equals 1.0 at baseline; drops linearly; 0.0 at 3x baseline
    lat_ratio = session_stats['avg_latency'] /
max(session_stats['baseline_latency'], 1)
    lat_score = max(0.0, 1.0 - (lat_ratio - 1.0) / 2.0)

    # Component 4 — Loop Frequency Score (20%)
    # -0.1 per extra loop above 1; floor at 0.0
    loop_score = max(0.0, 1.0 - (session_stats['avg_loop_count'] - 1.0) * 0.1)
```

```

    score = (0.40 * tool_sr + 0.20 * tok_eff + 0.20 * lat_score + 0.20 *
loop_score) * 100

    return {
        'score' : round(score, 2),
        'tool_success_rate' : round(tool_sr, 4),
        'token_efficiency' : round(tok_eff, 4),
        'latency_score' : round(lat_score, 4),
        'loop_frequency_score': round(loop_score, 4),
        'risk_level' : score_to_risk(score),
        'predicted_failure_prob': score_to_failure_prob(score),
    }

```

Score Bands

Score Range	Risk Level	Predicted Failure (next 10 calls)	Recommended Action
85 – 100	LOW	< 5%	Monitor normally; no action required
65 – 84	MEDIUM	5 – 20%	Watch closely; review recent tool failure logs
40 – 64	HIGH	20 – 50%	Investigate now; consider restarting session
0 – 39	CRITICAL	> 50%	Terminate session immediately; alert operator

6.6 Model Retraining Strategy (scheduler.py)

- Startup: 500 synthetic baseline events generated by simulator before real data exists — ensures model is ready from first real event.
- Periodic: APScheduler job every 30 min (MODEL_RETRAIN_INTERVAL_MINUTES). Uses last 5,000 events per agent (or all if fewer).
- Atomic swap: new model written to {path}.tmp, then os.replace() — zero downtime, no read during write.
- Fallback: if fewer than 200 events exist, threshold-based rules fire instead of IF (token > 3x rolling mean = spike, loop_count > 10 = loop).
- Model files stored in Docker volume ml_models:/app/models — persisted across container restarts.

7. Agent Simulator

The simulator generates realistic telemetry without LLM API calls. Critical for development, ML training data, and viva demonstration — anomalies can be injected live via the API.

7.1 Agent Behavior Profiles

Property	Customer Support A001	Research A002	Sales A003
Normal tokens / action	80 – 150	200 – 400	100 – 200
Normal latency (ms)	200 – 800	800 – 2 000	300 – 900
Normal loop count	1 (rarely 2)	1 – 3	1 – 2
Tool success rate	92 – 97%	80 – 90%	85 – 93%
Session token budget	2 000	8 000	4 000
Expected tokens / session	500 – 1 200	1 500 – 3 500	800 – 1 800
Cost / 1 k tokens (USD)	0.002	0.002	0.002
Tools pool	faq_lookup, ticket_create, escalate, send_email	web_search, doc_retrieve, summarize, cite	crm_lookup, lead_score, schedule_meeting, send_followup
Tick interval	2 s	3 s	2.5 s
Background anomaly rate	2% of events	5% of events	3% of events

7.2 Anomaly Injection Modes

Anomaly Type	What the Simulator Does	Expected Detection	Severity
token_spike	tokens_used set to 5x–15x normal for N events	IF score < -0.7 within 1 event	CRITICAL
infinite_loop	loop_count increments each event; status stays SUCCESS	IF score < -0.5 after 3+ loop events	WARNING → CRITICAL
high_latency	latency_ms set to 4x–10x baseline	IF score < -0.6 within 1 event	WARNING
tool_failure_cascade	status=FAILURE for 5+ consecutive events	Failure rate alert + IF score	CRITICAL
behavioral_drift	tokens slowly increase 5% per event over 20 events (gradual)	IF drift after 8–12 events	WARNING

7.3 Base Agent Architecture

```
# base_agent.py
import asyncio, httpx, random
from abc import ABC, abstractmethod

class BaseAgent(ABC):
    def __init__(self, agent_id: str, profile: dict, api_url: str, token: str):
        self.agent_id = agent_id
        self.profile = profile
```

```

        self.api_url      = api_url
        self.headers      = {'Authorization': f'Bearer {token}'}
        self.session_id   = None
        self.loop_count   = 0
        self._running     = False
        self._inject      = None    # set by anomaly_injector

    @abstractmethod
    def generate_event(self) -> dict:    # subclass returns realistic values
        ...

    async def run(self):
        self._running = True
        await self._start_session()
        while self._running:
            event = self.generate_event()
            if self._inject:                # override with anomaly values
                event = self._inject(event)
            await self._emit(event)
            await asyncio.sleep(self.profile['tick_interval'])

    async def _emit(self, event: dict):
        async with httpx.AsyncClient() as client:
            await client.post(f'{self.api_url}/telemetry/ingest',
                             json=event, headers=self.headers)

    def inject_anomaly(self, anomaly_fn):    # called by anomaly_injector.py
        self._inject = anomaly_fn

    def stop(self):
        self._running = False

```

7.4 Anomaly Injector

```

# anomaly_injector.py
def make_token_spike(multiplier=8, duration=3):
    count = {'n': 0}
    def inject(event):
        if count['n'] < duration:
            event['tokens_used'] = event['tokens_used'] * multiplier
            count['n'] += 1
        else:
            event['_restore'] = True    # signal to BaseAgent to clear inject
        return event
    return inject

# Similar factory functions for other anomaly types:
# make_infinite_loop(), make_high_latency(), make_failure_cascade(), make_drift()

```

8. Frontend Architecture

8.1 Pages

Page	Route	Key Components	Refresh
Login	/login	LoginForm	—
Dashboard	/	OverviewKPIs, AgentGrid, TokenChart, CostChart, AnomalyLog	5 s poll
Agent Detail	/agents/:id	ReliabilityGauge, LatencyChart, ToolSuccessBar, ToolGraph, SessionTable	3 s poll
Anomaly History	/anomalies	AlertTable, SeverityFilter, TypeFilter, AcknowledgeButton	10 s poll
Settings	/settings	AgentThresholdForm, BudgetEditor	On save

8.2 Component Descriptions

OverviewKPIs

Four stat cards in a row: (1) Active Agents — count of agents with at least one event in last 5 min. (2) Alerts Today — total anomaly_alerts created in last 24 h. (3) Avg Reliability Score — mean score across all active agents. (4) Cost Today — sum of total_cost_usd across all sessions started today.

AgentCard

Grid card per agent showing: name, type badge, current Reliability Score as large number with background color (green ≥ 85 , amber ≥ 65 , red < 65), risk_level badge, and last event timestamp. Clicking navigates to /agents/:id.

TokenChart

Recharts LineChart. One line per agent colored distinctly. X-axis: time (last 1 h by default, toggle to 6 h / 24 h). Y-axis: tokens_used per event. Horizontal ReferenceLine at each agent's token_budget threshold. Data source: GET /dashboard/metrics/token-usage.

ReliabilityGauge

Recharts RadialBarChart in semicircle mode. Shows 0–100 score with color zones: green (85–100), amber (65–84), red (0–64). Below the gauge: a 4-row table showing each component score and its weight. Below that: risk level badge and predicted failure probability text.

AnomalyLog

Auto-refreshing table. Columns: timestamp, agent name, alert_type (colored chip), severity (INFO=blue / WARNING=amber / CRITICAL=red), anomaly_score (2 dp), Acknowledge button. CRITICAL rows have a red left border. Plays a short audio beep on new CRITICAL alert using the Web Audio API.

ToolGraph

Directed graph of tool invocations in a session. Each node = one tool call. Edge direction = sequence. Node color: green (SUCCESS), red (FAILURE/TIMEOUT). Node size proportional to latency_ms. Rendered with Recharts Sankey or a lightweight custom SVG D3 layout. Data from GET /sessions/:id/tool-graph.

8.3 Data Fetching Hooks

```
// hooks/useAgents.js
import { useState, useEffect } from 'react';
import client from '../api/client';

export function useAgents(intervalMs = 5000) {
  const [agents, setAgents] = useState([]);
  const [error, setError] = useState(null);

  useEffect(() => {
    const fetch = () => {
      client.get('/dashboard/agents/active')
        .then(r => setAgents(r.data))
        .catch(e => setError(e));
    };
    fetch();
    const id = setInterval(fetch, intervalMs);
    return () => clearInterval(id);
  }, [intervalMs]);

  return { agents, error };
}

// Phase 2: replace setInterval with EventSource for SSE
// const es = new EventSource('/api/v1/events/stream?agent_id=all');
// es.onmessage = e => setAgents(JSON.parse(e.data));
```

8.4 Axios Client

```
// api/client.js
import axios from 'axios';

const client = axios.create({
  baseURL: process.env.REACT_APP_API_URL || 'http://localhost:8000/api/v1',
  timeout: 10000,
});

client.interceptors.request.use(cfg => {
  const token = localStorage.getItem('access_token');
  if (token) cfg.headers.Authorization = `Bearer ${token}`;
  return cfg;
});

client.interceptors.response.use(
  res => res,
  err => {
    if (err.response?.status === 401)
      window.location.href = '/login';
    return Promise.reject(err);
  }
);
```

```
| export default client;
```

9. Authentication & Security

9.1 JWT Flow

Step	Actor	Action
1	User	POST /auth/login with username + password
2	Backend	Verify bcrypt hash. On success: create access_token (30 min) + refresh_token (7 days) signed with SECRET_KEY using HS256
3	Frontend	Store both tokens in localStorage. Attach access_token to every request via Axios interceptor
4	Backend	Each protected route calls get_current_user() dependency — decodes JWT, loads user from DB
5	Frontend	On 401 response: call POST /auth/refresh with refresh_token. On success: swap tokens. On failure: redirect to /login

9.2 JWT Payload

```
# Access token payload
{
  "sub":      "user-uuid",    # user.id
  "username": "admin",
  "type":     "access",
  "iat":      1718000000,     # issued at (Unix)
  "exp":      1718001800     # expires at (iat + 1800 s)
}
```

9.3 Security Checklist

- SECRET_KEY must be generated with: openssl rand -hex 32 — never hardcode, always from env var.
- CORS: allow only http://localhost:3000 in development; set ALLOWED_ORIGINS env var for production.
- Passwords: hashed with bcrypt cost factor 12 via passlib.
- Phase 1: single role — all authenticated users have full access. Acceptable for demo/academic use.
- Phase 2: add roles column to users table — ADMIN (full access), OPERATOR (acknowledge alerts, config), VIEWER (read-only dashboard).

10. Deployment — Phase 1

10.1 docker-compose.yml

```
version: '3.8'

services:
  db:
    image: postgres:15
    restart: unless-stopped
    environment:
      POSTGRES_DB: Infera
      POSTGRES_USER: Infera
      POSTGRES_PASSWORD: ${DB_PASSWORD}
    volumes:
      - postgres_data:/var/lib/postgresql/data
    ports: ['5432:5432']
    healthcheck:
      test: ['CMD-SHELL', 'pg_isready -U Infera']
      interval: 10s
      timeout: 5s
      retries: 5

  backend:
    build: ./backend
    restart: unless-stopped
    environment:
      DATABASE_URL:
        postgresql+asyncpg://Infera:${DB_PASSWORD}@db:5432/Infera
      SECRET_KEY: ${SECRET_KEY}
      ACCESS_TOKEN_EXPIRE_MINUTES: 30
      REFRESH_TOKEN_EXPIRE_DAYS: 7
      MODEL_RETRAIN_INTERVAL_MINUTES: 30
      ANOMALY_CONTAMINATION_RATE: 0.05
      TOKEN_SPIKE_THRESHOLD: 3.0
      LATENCY_SPIKE_THRESHOLD: 3.0
      LOOP_COUNT_THRESHOLD: 10
      FAILURE_CASCADE_WINDOW: 5
      SIMULATOR_TICK_INTERVAL_SEC: 2
      COST_PER_1K_TOKENS: 0.002
    ports: ['8000:8000']
    depends_on:
      db: { condition: service_healthy }
    volumes:
      - ml_models:/app/models

  frontend:
    build: ./frontend
    restart: unless-stopped
    environment:
      REACT_APP_API_URL: http://localhost:8000/api/v1
    ports: ['3000:80']
    depends_on: [backend]

volumes:
  postgres_data:
  ml_models:
```

10.2 Environment Variables (.env.example)

Variable	Example Value	Description
DB_PASSWORD	changeme123	PostgreSQL password — change before any shared deployment
SECRET_KEY	\$(openssl rand -hex 32)	JWT signing key — generate fresh for each deployment
ACCESS_TOKEN_EXPIRE_MINUTES	30	JWT access token lifetime in minutes
REFRESH_TOKEN_EXPIRE_DAYS	7	JWT refresh token lifetime in days
MODEL_RETRAIN_INTERVAL_MINUTES	30	How often APScheduler retrains the IF model
ANOMALY_CONTAMINATION_RATE	0.05	IF contamination param; tune based on observed false positive rate
TOKEN_SPIKE_THRESHOLD	3.0	Z-score above which a token event is flagged as a spike
LATENCY_SPIKE_THRESHOLD	3.0	Z-score above which a latency event is flagged
LOOP_COUNT_THRESHOLD	10	loop_count above this triggers CRITICAL alert directly
FAILURE_CASCADE_WINDOW	5	Consecutive FAILURE events that trigger cascade alert
SIMULATOR_TICK_INTERVAL_SEC	2	Seconds between simulated agent events
COST_PER_1K_TOKENS	0.002	USD cost used in cost estimation (adjust to model pricing)

10.3 Commands

```
# One-time: generate a SECRET_KEY
openssl rand -hex 32

# Copy env template
cp .env.example .env # then fill in DB_PASSWORD and SECRET_KEY

# Build and start everything
docker-compose up -d --build

# Apply database migrations (first run only)
docker-compose exec backend alembic upgrade head

# Seed initial admin user (run once after migration)
docker-compose exec backend python -m app.scripts.seed_admin

# View backend logs
docker-compose logs -f backend

# Stop (keeps data volumes)
docker-compose down

# Full reset — destroys database and model files
docker-compose down -v

# Open Swagger UI
# http://localhost:8000/docs
```

```
# Open React dashboard
# http://localhost:3000
```

10.4 Viva Demo Script (~10 minutes)

Rehearse this script at least twice before the viva. Inject anomalies during the demo — live detection is the most impressive moment.

Step	Action	What the Examiner Sees
1	<code>docker-compose up -d</code> (already running). Open <code>localhost:3000</code> .	Login page. Log in as admin.
2	Dashboard loads.	Three AgentCards — all GREEN, scores 85+. Token chart flat and healthy.
3	Open Swagger at <code>localhost:8000/docs</code> . Show the auto-generated API docs.	Examiner sees full API specification — demonstrates FastAPI professionalism.
4	Call <code>POST /simulator/inject-anomaly</code> — <code>token_spike</code> on A001, <code>duration=5</code> .	A001 token chart spikes sharply. Within 5 seconds, CRITICAL alert appears in AnomalyLog.
5	Point to A001 AgentCard.	Card has turned RED. Reliability Score dropped from ~90 to ~40.
6	Click A001 to open Agent Detail page.	ReliabilityGauge shows CRITICAL band. Component breakdown shows <code>token_efficiency</code> near zero.
7	Call <code>POST /simulator/inject-anomaly</code> — <code>infinite_loop</code> on A002.	A002 <code>loop_count</code> increments each refresh. WARNING alert fires, then escalates to CRITICAL.
8	Acknowledge one alert from the AnomalyLog.	Alert moves to acknowledged state; badge count decreases.
9	Open <code>GET /ml/model/stats</code> in Swagger.	Show <code>trained_on_count</code> , <code>contamination</code> , <code>f1_score</code> — demonstrates real ML model in production.
10	<code>POST /simulator/stop</code> . Show final session summaries.	Sessions show total tokens, USD cost, final Reliability Score per agent.

11. Phase 2 — Architecture & Migration Plan

Phase 2 evolves Infera from a single-node system to a distributed, enterprise-scale platform. The core ML logic, database schema, and API contracts are preserved — only the transport and infrastructure layers change.

11.1 What Stays the Same

- PostgreSQL schema — all 7 tables, all indexes, all Alembic migrations carry forward unchanged.
- FastAPI routes — same URL paths and response schemas; only the ingestion internals change (DB write becomes Kafka publish).
- ML pipeline — Isolation Forest, LOF, Reliability Score, and `model_store.py` unchanged.
- React frontend — extend existing components; replace polling hooks with SSE hooks.
- JWT authentication — same implementation; add roles column to users table.

11.2 What Changes

Phase 1	Phase 2 Replacement	Reason
POST <code>/telemetry/ingest</code> writes directly to PostgreSQL	Publishes to Kafka topic <code>Infera.telemetry.raw</code>	Decouples agent from DB write latency
ML scoring runs synchronously in the request handler	Kafka consumer scores events, writes to DB, publishes to scored topic	Non-blocking; handles burst load
Dashboard polls REST endpoints every 5 s	Frontend subscribes to SSE endpoint <code>/events/stream</code>	True real-time, lower network overhead
No external notification	Notification service consumes <code>Infera.alerts</code> , sends Slack/email	Enterprise alerting
No infrastructure metrics	Prometheus scrapes <code>/metrics</code> ; Grafana dashboards	Operational visibility at scale

11.3 Kafka Topics

Topic	Publisher	Consumer	Message Schema
<code>Infera.telemetry.raw</code>	Agent SDK / Simulator	ML Consumer Service	Raw TelemetryEvent JSON
<code>Infera.telemetry.scored</code>	ML Consumer	SSE Service / Dashboard	TelemetryEvent + <code>anomaly_score</code> + <code>is_anomaly</code>
<code>Infera.alerts</code>	ML Consumer (on anomaly)	Notification Service	AnomalyAlert JSON — triggers email/Slack
<code>Infera.sessions</code>	Agent SDK	Session Manager	Session start/end events

11.4 New `docker-compose.yml` Services (Phase 2 additions)

```

zookeeper:
  image: confluentinc/cp-zookeeper:7.5.0
  environment: { ZOOKEEPER_CLIENT_PORT: 2181 }

kafka:
  image: confluentinc/cp-kafka:7.5.0
  depends_on: [zookeeper]
  environment:
    KAFKA_ZOOKEEPER_CONNECT:          zookeeper:2181
    KAFKA_ADVERTISED_LISTENERS:      PLAINTEXT://kafka:9092
    KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR: 1
    KAFKA_AUTO_CREATE_TOPICS_ENABLE:  true
  ports: ['9092:9092']

kafka-consumer:
  build: ./backend          # same image, different CMD
  command: python -m app.kafka.consumer
  environment:
    KAFKA_BOOTSTRAP_SERVERS: kafka:9092
    DATABASE_URL: postgresql+asyncpg://Infera:${DB_PASSWORD}@db:5432/Infera
  depends_on: [kafka, db]

redis:
  image: redis:7-alpine
  ports: ['6379:6379']

prometheus:
  image: prom/prometheus:latest
  volumes:
    - ./monitoring/prometheus.yml:/etc/prometheus/prometheus.yml
  ports: ['9090:9090']

grafana:
  image: grafana/grafana:latest
  ports: ['3001:3000']
  environment: { GF_SECURITY_ADMIN_PASSWORD: '${GRAFANA_PASSWORD}' }
  volumes:
    - ./monitoring/grafana/dashboards:/etc/grafana/provisioning/dashboards

```

11.5 Python SDK Design (Phase 2)

Allows real LangChain or custom agents to auto-instrument with minimal code.

```

pip install Infera-sdk

# --- Option 1: Decorator (wraps any tool function) ---
from Infera import trace

@trace(agent_id='A001', session_id='S1001',
       api_url='http://Infera:8000', token='Bearer ...')
def web_search(query: str) -> str:
    ... # your tool logic

# --- Option 2: Context manager (full session) ---
from Infera import Session

with Session(agent_id='A001') as s:
    result = s.tool_call('web_search', query='AI safety')
    # Emits telemetry automatically: latency, tokens estimated, status

# --- Option 3: LangChain callback ---
from Infera.integrations.langchain import InferaCallback
from langchain.agents import AgentExecutor

```

```
executor = AgentExecutor(callbacks=[InferaCallback(agent_id='A001')])
```

11.6 New API Endpoints in Phase 2

Endpoint	Description
GET /metrics	Prometheus metrics endpoint. Exposes agent reliability scores as gauges, alert counts as counters.
GET /events/stream?agent_id=all	Server-Sent Events stream. Pushes scored telemetry events to React dashboard in real time.
POST /webhooks/register	Register an external URL to receive POST calls on CRITICAL alerts (Slack incoming webhook, PagerDuty, etc.).
GET /sdk/config/{agent_id}	Returns SDK configuration for an agent: API URL, agent thresholds, API key for Kafka auth.
GET /topology	Returns agent-to-agent call graph for multi-agent pipeline visualisation.

11.7 Phase 2 Development Checklist

- Add Kafka + Zookeeper to Docker Compose; verify topics auto-create on first publish.
- Refactor POST /telemetry/ingest to publish to Kafka instead of direct DB write.
- Write kafka/consumer.py: read raw events, call feature_engineering, call IF model, write to DB, publish to scored topic.
- Add Redis: cache dashboard aggregates with 5 s TTL; eliminate per-request DB aggregation queries.
- Replace React polling hooks with useEventSource hooks consuming /events/stream SSE.
- Add /metrics endpoint using prometheus-client: Gauge per agent for reliability score, Counter for alerts.
- Configure prometheus.yml to scrape backend:8000/metrics every 15 s.
- Import Grafana dashboard JSON for: Kafka consumer lag, DB query latency, per-agent score trend.
- Build Infera-sdk Python package: trace decorator, Session context manager, InferaCallback for LangChain.
- Add roles column to users table (Alembic migration): ADMIN, OPERATOR, VIEWER.
- Write Slack webhook integration: on CRITICAL alert, POST to registered Slack webhook URL.
- Conduct load test targeting 1 000 events/s through Kafka; document throughput results for paper.
- Write Kubernetes manifests: Deployment + Service + ConfigMap + Secret for each component.

12. Research Paper Plan

Field	Value
Title	Behavioral Drift Detection in Multi-Step LLM Agents Through Telemetry Analytics
Alt Title	Infera: A Real-Time Observability Framework for Autonomous AI Agents
Target Venue	IEEE ICACCS 2027 (International Conference on Advanced Computing and Communication Systems) or Springer LNNS
Status	Written in Semester 8 after Phase 2 evaluation data is collected
Authors	Your name + Guide name
Keywords	LLM agents, anomaly detection, Isolation Forest, agent observability, behavioral drift, telemetry analytics

12.1 Abstract Draft

Replace all bracketed values with real experimental results. This is a structural draft only.

Autonomous AI agents powered by Large Language Models exhibit non-deterministic behavior patterns that render traditional application performance monitoring tools inadequate. In this work we present Infera, an open-source observability platform that captures execution telemetry from LLM agents and applies unsupervised machine learning to detect behavioral anomalies in real time. We propose the Agent Reliability Score (ARS), a composite metric aggregating tool success rate, token efficiency, response latency, and loop frequency into a single predictive health index updated after every agent action. Applying Isolation Forest to a 10-dimensional telemetry feature space, our system detects token consumption spikes, infinite reasoning loops, tool failure cascades, and behavioral drift with a precision of [P]%, recall of [R]%, and F1 of [F] on a labeled evaluation dataset of [N] injected anomaly events across three agent archetypes. Detection latency averages [D] ms from event emission to dashboard alert. We further demonstrate horizontal scalability from single-node to Kafka-based distributed ingestion, sustaining [T] events/second with sub-[L] ms scoring latency. Infera is publicly available and provides both a production monitoring platform and a reproducible baseline for future research in AI agent observability.

12.2 Paper Structure

Section	Content	Target Length
1. Introduction	Problem (3 para): why LLM agents are unmonitorable. Contributions (3 bullets). Paper structure.	0.5 pages
2. Related Work	APM tools (Datadog, New Relic) — designed for deterministic software. LLM eval frameworks (RAGAS, TruLens, LangSmith) — offline eval, not runtime. Anomaly detection literature (IF, LOF, LSTM-AD) — applied to network/system, not LLM agents. Gap statement.	1 page
3. System Architecture	Architecture diagram (Phase 1 single-node, Phase 2 distributed). Component table. Data flow description. Telemetry schema.	1 page

Section	Content	Target Length
4. ML Approach	Feature vector table (10 features). IF configuration. LOF baseline. Training strategy (contamination rate, min events). Score-to-severity mapping.	1 page
5. Agent Reliability Score	Formula derivation. Weight justification (cite existing APM literature for why tool success is most predictive). Score band table. Prediction mechanism.	0.5 pages
6. Evaluation Setup	Experimental setup: 3 agent archetypes, 5 anomaly types, [N] total events. Injected anomaly labeling method. Train/test split.	0.5 pages
7. Results	Table: Precision / Recall / F1 per anomaly type for IF vs LOF vs threshold baseline. Detection latency CDF plot. ARS correlation with actual failure rate. Phase 2 throughput benchmark.	1.5 pages
8. Discussion	Key findings. Limitations: simulated agents (discuss generalisability). Comparison with LangSmith and existing tools.	0.5 pages
9. Conclusion	Summary of 3 contributions. Future work: streaming ML, real agent integration, multi-tenant deployment.	0.25 pages
References	20–30 citations. Include: Isolation Forest original (Liu 2008), LOF (Breunig 2000), LangChain paper, LangSmith blog, APM survey paper.	1 page

12.3 Evaluation Results to Collect

- Precision, Recall, F1 for Isolation Forest — per anomaly type and overall.
- Same metrics for LOF (baseline comparison) and simple threshold rules (lower baseline).
- Detection latency: time from anomaly injection to alert in DB (target < 2 000 ms).
- False positive rate under normal operation over 2+ hours (target < 2%).
- Reliability Score correlation: compare score at event N against actual failure rate in events N+1 to N+10.
- Phase 2 throughput: events/second sustained through Kafka before consumer lag increases.
- Model retraining time vs training set size (50, 100, 200, 500, 1000, 5000 events).

12.4 Research Diary — Track These From Day 1

Document these as they happen. You will forget by Semester 8 and the paper will suffer.

- Date + rationale for every technology choice that was changed from the original plan.
- All IF hyperparameter experiments: contamination rates tried, resulting precision/recall for each.
- Baseline performance numbers BEFORE any optimization (for the 'improvement' story in the paper).
- Every false positive and false negative observed — what caused it, how you addressed it.

- Git commit hash for each major milestone (initial working API, first ML model, first dashboard).
- Any edge cases in anomaly detection that surprised you — these become 'interesting findings' in the Discussion section.

13. Development Notes, Decisions & Known Limitations

13.1 Phase 1 Known Limitations

Limitation	Impact	Phase 2 Fix
5-second polling dashboard	1–5 s stale data; acceptable for demo	Replace with SSE endpoint + useEventSource hook
IF needs 200+ events to train	First 200 events use fallback threshold rules	Synthetic seed data generated at startup solves this
Single-user only	No multi-tenancy; fine for academic demo	Add roles column; RBAC in Phase 2
No webhook notifications	Alerts only appear in dashboard	Notification service in Phase 2 adds Slack/email
SQLAlchemy async + sync mixing	Can cause runtime errors if not careful	Audit all DB calls in Phase 1; fix before Phase 2 refactor
Model files overwritten in-place	Brief window where model is inconsistent	Atomic os.replace() implemented in model_store.py

13.2 Likely Examiner Criticisms — Prepare These Answers

Criticism	Prepared Answer
Why simulated agents, not real LLMs?	Simulated agents produce labeled anomaly data — essential for precision/recall evaluation. Real LLMs are non-deterministic: you cannot reliably measure recall (some injected anomalies would be masked by natural variation). Simulation is standard practice in anomaly detection research.
Isn't Isolation Forest too simple?	IF is appropriate because: (1) no labeled data is available in production — unsupervised is mandatory; (2) the paper includes LOF comparison to empirically justify the choice; (3) IF has $O(n \log n)$ training time — important for 30-minute retraining cycles on live data.
What makes this novel?	Three novelties: (1) Agent Reliability Score — no prior composite metric exists for LLM agent health; (2) standardised telemetry schema for LLM agents — not defined anywhere in existing literature; (3) first open-source platform purpose-built for LLM agent observability (LangSmith is proprietary).
Why not WebSockets in Phase 1?	Polling was an intentional architectural decision documented in the design: it reduces frontend complexity and eliminates connection management overhead. It is explicitly listed as a Phase 2 upgrade with SSE, demonstrating forward architectural thinking rather than technical limitation.
Why FastAPI and not Spring Boot?	FastAPI enables the ML pipeline to run in-process without a separate microservice — this reduces system complexity by one component and was appropriate for Phase 1 throughput. Spring Boot is evaluated for Phase 2 if Java ecosystem tooling becomes advantageous for Kafka consumer scaling.

13.3 Semester 7 Timeline

Weeks	Milestone	Deliverable / Exit Criterion
1 – 2	Foundation	PostgreSQL schema created, Alembic migrations running, FastAPI app skeleton with health endpoint, Docker Compose starts all services
3 – 4	Auth + Agents API	POST /auth/login working with JWT, GET/POST /agents/ working, all schemas validated with Pydantic
5 – 6	Simulator	All three agents generating telemetry, POST /telemetry/ingest storing events, sessions opening and closing correctly
7 – 9	ML Pipeline	Feature engineering function complete, IF model trains on 500+ synthetic events, /telemetry/ingest returns anomaly_score, Reliability Score written to DB after each event
10 – 11	Alerts + Dashboard	Alert generation working, GET /dashboard/* endpoints returning correct data, AnomalyLog shows alerts in real time
12 – 13	React Frontend	All dashboard pages complete, TokenChart + ReliabilityGauge + AnomalyLog functional, anomaly injection demo working end-to-end
14 – 15	Evaluation	200+ injected anomalies evaluated, precision/recall/F1 computed, IF vs LOF comparison table, benchmark numbers documented in research diary
16	Submission	Docker Compose one-command deployment verified on a clean laptop, project report submitted, viva demo script rehearsed twice

13.4 requirements.txt (Backend)

```
fastapi==0.111.0
uvicorn[standard]==0.29.0
sqlalchemy[asyncio]==2.0.29
asyncpg==0.29.0
alembic==1.13.1
pydantic-settings==2.2.1
python-jose[cryptography]==3.3.0
passlib[bcrypt]==1.7.4
scikit-learn==1.4.2
pandas==2.2.2
numpy==1.26.4
joblib==1.4.0
apscheduler==3.10.4
httpx==0.27.0
python-multipart==0.0.9    # for form data in login
```

13.5 package.json key dependencies (Frontend)

```
{
  "dependencies": {
    "react": "^18.3.1",
    "react-router-dom": "^6.23.1",
    "axios": "^1.7.2",
    "recharts": "^2.12.7",
    "react-dom": "^18.3.1"
  }
}
```